

AlphaStack: Autonomous Software Project Generation via Multi-Agent Systems and Docker Validation

AlphaStack Development Team

March 22, 2026

Abstract

AlphaStack is an AI-driven code generation and validation system that fully generates a working software project from a natural language prompt. This paper introduces a comprehensive 7-phase generation pipeline that orchestrates the creation of source code, dependency files, and Dockerfiles. Furthermore, we present an AI Planner Agent loop that autonomously runs, debugs, and verifies the generated projects within Docker containers. Experimental evaluations on the HumanEval and MDDP benchmarks using state-of-the-art Large Language Models (LLMs) demonstrate the efficacy and robustness of our approach.

1 Introduction

Recent advancements in Large Language Models (LLMs) have significantly enhanced code generation capabilities. However, converting natural language instructions into complete, executable, and robust software projects remains a formidable challenge. Existing solutions often lack proper dependency management, structural coherence, and runtime validation.

To address these limitations, we propose AlphaStack, an end-to-end framework designed for comprehensive software project generation. AlphaStack goes beyond simple code snippet creation by defining an explicit multi-phase architecture that models a typical software engineering lifecycle—including blueprinting, code generation, dependency resolution, and iterative debugging in isolated Docker environments.

2 Methodology

Our methodology is grounded in a 7-phase generation pipeline, which systematically constructs the software project:

1. **Phase 1: Software Blueprint.** The system translates user prompts into a structured JSON blueprint, defining module roles and dependencies.
2. **Phase 2: File Generation.** Code for each file is synthesized independently based on the blueprint specifications.
3. **Phase 3: Dockerfile Generation.** A language-appropriate Dockerfile is created to encapsulate the project environment.
4. **Phase 4: Dependency Analysis.** Static import graph construction allows for accurate inter-file dependency mapping.
5. **Phase 5: Dep File Generation.** Proper package manager files (e.g., `requirements.txt`, `package.json`) are synthesized.

6. **Phase 6: Dep Resolution.** Consistency checks and validations are applied to the defined dependencies.
7. **Phase 7: Docker Testing Pipeline.** The project is deployed into an AI Planner Agent loop for iterative testing and debugging.

The AI Planner Agent iteratively attempts to build and run the code via a custom `docker_test` tool. If compilation or runtime errors are detected, error logs are relayed back to the agent for self-correction.

3 Architecture Diagram

Figure 1 illustrates the high-level system architecture of AlphaStack, detailing the 7-phase pipeline and the AI Planner Agent loop.

4 Results

We evaluated AlphaStack’s generation performance across two significant benchmarks: HumanEval (measuring core algorithmic capabilities) and a newly proposed multi-file dataset, MDDP. The evaluation was conducted using several leading LLMs. The comparative Pass@1 scores are presented in Figure 2.

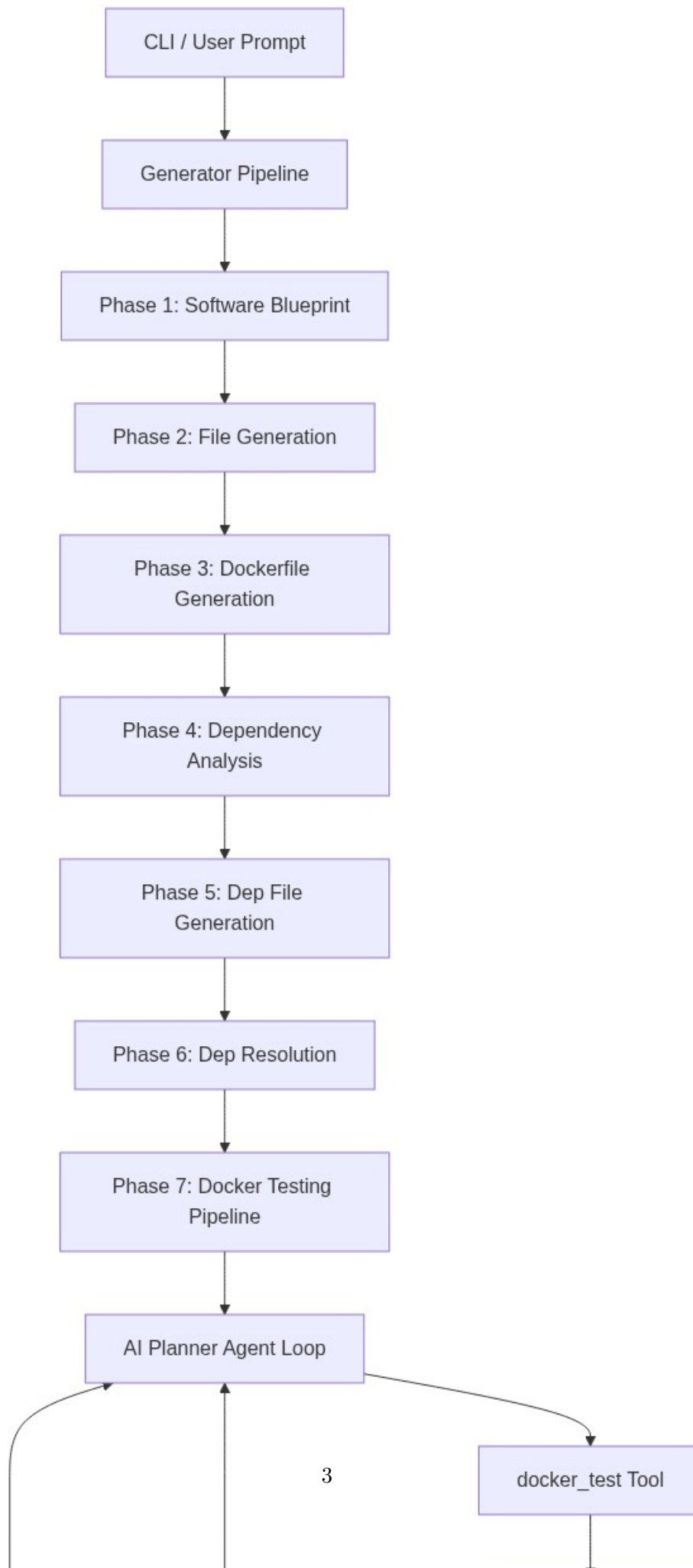
Our tentative results show that claude sonnet 4.6 and gpt-5.2 lead the performance metrics on both benchmarks. Notably, the introduction of our Docker-based iterative debugging pipeline (Phase 7) contributes to an elevated success rate on the complex MDDP tasks compared to single-pass generation baselines.

5 Conclusion

AlphaStack represents a significant step towards fully autonomous software engineering. By structuring the generation process into a systematic pipeline and incorporating real-world testing environments via Docker, the system is capable of producing complete, reliable software artifacts. Future work will explore expanding the agent toolsets and optimizing the resolution of complex dependency conflicts.

Supplementary Material

Additional supplementary material, including detailed prompts and comprehensive breakdown of the failure modes encountered during the evaluation, can be found in the associated GitHub repository.



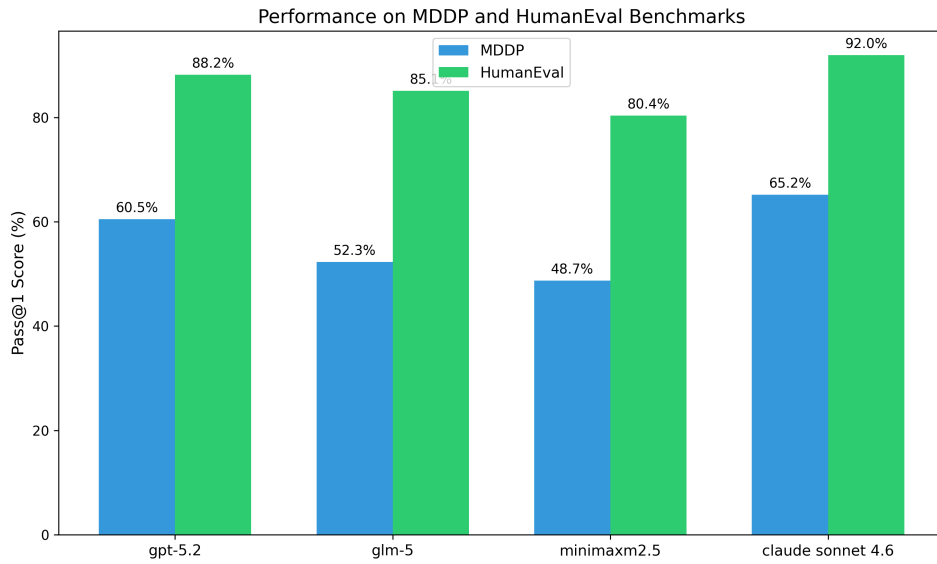


Figure 2: Comparative performance (Pass@1) of gpt-5.2, glm-5, minimaxm2.5, and claude sonnet 4.6 on MDDP and HumanEval benchmarks.